DEPARTMENT OF
COMPUTER SCIENCE & ENGINEERING
Discover. Learn. Empower.

Assignment

Student Name: Priyanshu Choudhary	**UID: 23BCS12648**
Branch: BE CSE	**Section/Group: KRG_3B**
Semester: 6th	**Date of Performance: 27/02/26**
Subject Name: Full Stack - II	**Subject Code: 23CSH-309**

1. Aim

To build a Water Intake Tracker inside a React application for the EcoTrack dashboard.

2. Objectives

To create a dashboard page where a logged-in user can navigate to the tracker, increase/decrease their water count, save a daily goal, fetch a health tip from an external API, and prevent unnecessary component re-renders.

3. Tech Stack

- **Frontend Library**: React.js
- **Routing**: React Router (react-router-dom)
- **State Management**: React Hooks (useState, useEffect)
- **Performance Optimization**: React.memo, useCallback
- **Storage**: Browser localStorage
- **API Integration**: JavaScript fetch API

4. Procedure

- **Step 1: Routing Setup**
  - Configured React Router to handle three main paths: /login for the Login page, /dashboard for the main Dashboard, and /dashboard/water for the Water Tracker.

- Implemented a Protected Route wrapper to check for authentication; if a user is not logged in, they are automatically redirected to the /login route.
- Created a mock login function that stores a fake token in localStorage.

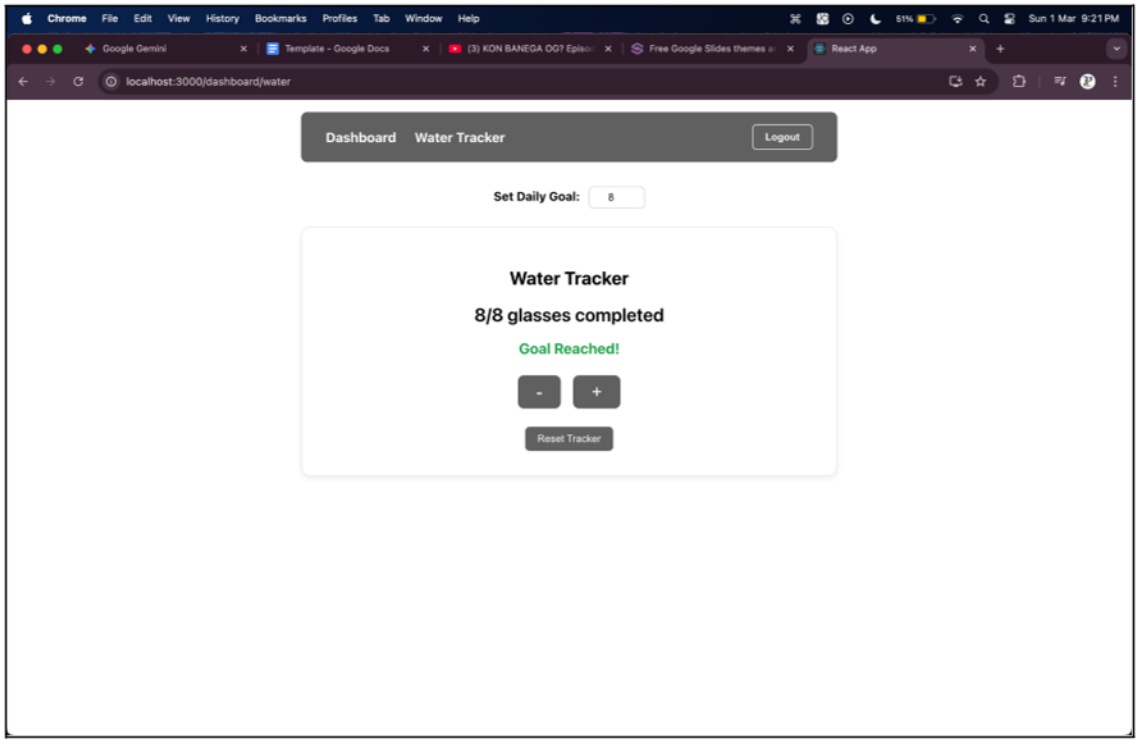

FIG: WATER TRACKER

- **Step 2: State Management**
  - Inside the WaterTracker page, initialized state variables for count (defaulting to 0) and goal (defaulting to 8).
  - Developed functions to add water, remove water, and a reset button to clear the progress.
  - Utilized useEffect to save the water count in localStorage and load the previous value whenever the page is refreshed.
- **Step 3: API Integration**
  - Set up a useEffect hook to fetch a daily health tip from https://api.adviceslip.com/advice on component mount.
  - Implemented UI states to display "loading text" while the fetch is in progress, and basic "error handling" if the API request fails.
  - Displayed the fetched data under the heading "Today's Health Tip:".
- **Step 4: Performance Optimization**

    - Extracted the counter UI into a separate component named <CounterDisplay />.
    - Wrapped the <CounterDisplay /> in React.memo to prevent it from re-rendering unnecessarily.
    - Wrapped the add, remove, and reset handler functions in useCallback to maintain referential equality across renders.
- **Step 5: User Interface (UI)**
    - Built a clean layout including a Navbar with links to the Dashboard, Water Tracker, and a Logout button.
    - Designed a Tracker card that displays buttons and the progress text in a "X/Y glasses completed" format.
    - Added conditional rendering to display a "Goal Reached" message when the user's count is greater than or equal to their goal

5. Learning Outcomes

The React application successfully runs a secure, multi-page dashboard. The user can log in, navigate to the Water Tracker, and log their daily hydration. The counter updates dynamically, persists data across browser reloads, and effectively prevents unnecessary DOM paints when unrelated state variables update.

6.Summary

The "Daily Water Tracker" feature ticket was fully resolved. By combining React Router for navigation, local storage for data persistence, and advanced hooks (useCallback, React.memo) for performance optimization, the resulting feature is lightweight, functional, and meets all product team requirements.